Digital Logic Design

A Complete Reference

Topics covered: Number systems, Boolean algebra
Logic gates, Combinational circuits
Karnaugh maps, Sequential circuits
Flip-flops, Finite state machines

April 2026

Contents

1 Number Systems **3**
1.1 Positional Notation . . . 3
1.2 Binary Arithmetic . . . 3
1.2.1 Addition Rules . . . 3
1.2.2 2's Complement Representation . . . 3
1.3 Number Conversion Summary . . . 3

2 Boolean Algebra **4**
2.1 Fundamental Operations . . . 4
2.2 Boolean Identities . . . 4
2.3 Theorems . . . 4
2.4 De Morgan's Laws . . . 4
2.5 Algebraic Simplification Example . . . 4

3 Logic Gates **5**
3.1 Basic Gates and Truth Tables . . . 5
3.2 Gate Symbols (IEEE/ANSI) . . . 5

4 Combinational Circuits **5**
4.1 Half Adder . . . 5
4.2 Full Adder . . . 5
4.3 Multiplexer (MUX) . . . 6
4.4 Decoder . . . 6
4.5 Encoder and Priority Encoder . . . 6

5 Karnaugh Maps **6**
5.1 Structure and Gray Code Ordering . . . 6
5.2 Grouping Rules . . . 7
5.3 Reading Groups to Product Terms . . . 7
5.4 Example: 3-Variable K-map . . . 7

6 Sequential Circuits **7**
6.1 Combinational vs Sequential Comparison . . . 8
6.2 Synchronous vs Asynchronous . . . 8
6.3 Registers and Counters . . . 8

7 Flip-Flops **8**
7.1 SR Latch (Level-Sensitive) . . . 8
7.2 D Flip-Flop . . . 8
7.3 JK Flip-Flop . . . 9
7.4 T Flip-Flop . . . 9
7.5 Flip-Flop Excitation Tables . . . 9

8 Finite State Machines **9**
8.1 Moore vs Mealy Machines . . . 10
8.2 Example: Traffic Light Controller (Moore) . . . 10
8.3 FSM Design Procedure . . . 10

9 Timing and Hazards **10**
9.1 Propagation Delay 10
9.2 Setup and Hold Times 11
9.3 Hazards 11

10 Quick Reference **11**
10.1 Powers of 2 11
10.2 Key Formulas 11

Number Systems

Digital systems represent all data using discrete voltage levels mapped to binary digits (bits). Understanding number bases is foundational to digital design.

Positional Notation

The value of a number in base r is:

$$N = d_{n-1} \cdot r^{n-1} + d_{n-2} \cdot r^{n-2} + \cdots + d_1 \cdot r^1 + d_0 \cdot r^0$$

Table 1: Common number bases in digital systems

Base	Name	Digits	Example
2	Binary	0, 1	$1011_2 = 11_{10}$
8	Octal	0–7	$52_8 = 42_{10}$
10	Decimal	0–9	42_{10}
16	Hexadecimal	0–9, A–F	$2A_{16} = 42_{10}$

Binary Arithmetic

Addition Rules

Table 2: Single-bit binary addition

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

2's Complement Representation

The standard method for representing signed integers in hardware:

1. Represent the positive magnitude in binary
2. Invert all bits (1's complement)
3. Add 1 to the result

> **Example:** Represent -5 in 4-bit 2's complement
> $+5 = 0101_2 \quad \rightarrow \quad$ invert: $1010_2 \quad \rightarrow \quad$ add 1: $1011_2 = -5$

The range of an n-bit 2's complement number is -2^{n-1} to $2^{n-1} - 1$.

Number Conversion Summary

$$1011_2 = 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0 = 8 + 0 + 2 + 1 = 11_{10}$$

$$42_{10} = 32 + 8 + 2 = 2^5 + 2^3 + 2^1 = 101010_2$$

Boolean Algebra

Boolean algebra provides the mathematical foundation for analyzing and designing digital circuits. All variables take values in $\{0, 1\}$.

Fundamental Operations

Table 3: Three primitive Boolean operations

Operation	Symbol	Expression	Description
AND	$\cdot$	$F = A \cdot B$	1 only when all inputs are 1
OR	$+$	$F = A + B$	1 when any input is 1
NOT	$\bar{}$	$F = \bar{A}$	Complement of input

Boolean Identities

Table 4: Fundamental identities

OR form	AND form
$A + 0 = A$	$A \cdot 1 = A$
$A + 1 = 1$	$A \cdot 0 = 0$
$A + A = A$	$A \cdot A = A$
$A + \bar{A} = 1$	$A \cdot \bar{A} = 0$
$\bar{\bar{A}} = A$	(double negation)

Theorems

Commutative $A + B = B + A$ and $A \cdot B = B \cdot A$

Associative $(A + B) + C = A + (B + C)$ and $(A \cdot B) \cdot C = A \cdot (B \cdot C)$

Distributive $A \cdot (B + C) = A \cdot B + A \cdot C$

Absorption $A + A \cdot B = A$ and $A \cdot (A + B) = A$

De Morgan's Laws

> **Most important theorems in digital design.** To apply: flip the operator (AND↔OR) and complement every variable.
>
> $$\overline{A + B} = \bar{A} \cdot \bar{B} \quad \text{and} \quad \overline{A \cdot B} = \bar{A} + \bar{B}$$

Algebraic Simplification Example

Simplify $F = A \cdot B + A \cdot \bar{B}$:

$$F = A \cdot B + A \cdot \bar{B} = A \cdot (B + \bar{B}) = A \cdot 1 = \boxed{A}$$

Logic Gates

Logic gates are the physical implementations of Boolean operations, built from transistors.

Basic Gates and Truth Tables

Table 5: Truth tables for fundamental gates

A	B	NOT A	AND	OR	NAND	NOR	XOR	XNOR
0	0	1	0	0	1	1	0	1
0	1	1	0	1	1	0	1	0
1	0	0	0	1	1	0	1	0
1	1	0	1	1	0	0	0	1

Gate Symbols (IEEE/ANSI)

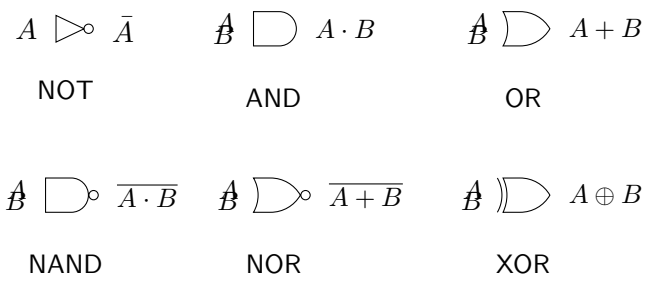

Figure 1: Standard US logic gate symbols

Universal gates: NAND and NOR are each individually sufficient to implement any Boolean function. Any AND, OR, or NOT can be built from only NAND gates (or only NOR gates). This is why CMOS circuits are predominantly built from NAND/NOR primitives.

Combinational Circuits

In a combinational circuit, outputs are determined solely by the current inputs — there is no internal state or memory.

Half Adder

Adds two single bits. Outputs: Sum and Carry.

$$\text{Sum} = A \oplus B \qquad \text{Carry} = A \cdot B$$

Full Adder

Extends the half adder by including a carry-in (C_{in}). Full adders are chained to build multi-bit ripple-carry adders.

$$\text{Sum} = A \oplus B \oplus C_{in}$$
$$C_{out} = A \cdot B + C_{in} \cdot (A \oplus B)$$

Table 6: Half adder truth table

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Table 7: Full adder truth table

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Multiplexer (MUX)

A multiplexer selects one of 2^n input lines based on n select lines.

2-to-1 MUX:

$$Y = \bar{S} \cdot I_0 + S \cdot I_1$$

General 2^n-to-1 MUX: For select bits $S_{n-1} \dots S_0$, the output is the input I_k where k is the decimal value of the select bits.

Decoder

An n-to-2^n decoder asserts exactly one output high for each input combination (one-hot encoding). Used extensively in memory address decoding.

2-to-4 decoder outputs:

$$Y_0 = \bar{A}\bar{B}, \quad Y_1 = \bar{A}B, \quad Y_2 = A\bar{B}, \quad Y_3 = AB$$

Encoder and Priority Encoder

An encoder performs the inverse of a decoder: 2^n inputs to n outputs. A *priority encoder* resolves the case where multiple inputs are active simultaneously by selecting the highest-priority (typically highest-index) active input.

Karnaugh Maps

A Karnaugh map (K-map) is a visual tool for minimizing Boolean expressions by grouping adjacent minterms. It eliminates the need for tedious algebraic manipulation.

Structure and Gray Code Ordering

Variables are arranged so adjacent cells differ in exactly one variable (Gray code order):

- 2-variable K-map: 2×2 grid

- 3-variable K-map: 2×4 grid
- 4-variable K-map: 4×4 grid

Grouping Rules

1. Groups must contain $1, 2, 4, 8, \ldots$ (powers of 2) cells
2. All cells in a group must contain 1
3. Groups may wrap around edges of the map
4. Use the fewest, largest possible groups
5. Every 1 must be covered by at least one group

Reading Groups to Product Terms

For each group, identify variables that remain constant throughout. Variables that change within the group cancel out of the term.

Example: 3-Variable K-map

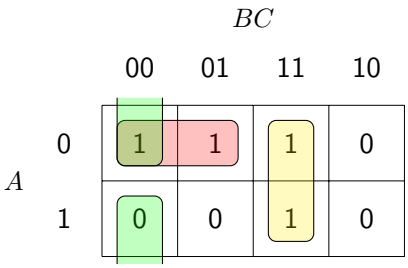

Figure 2: Example 3-variable K-map with minterms at 0, 1, 3, 7

The simplified expression for minterms $\{0, 1, 3, 7\}$:

$$F = \bar{A}\bar{B} + \bar{A}\bar{C} + BC$$

Further simplified:

$$F = \bar{A}(\bar{B} + \bar{C}) + BC = \overline{AB + AC} \cdot \overline{\ } + BC$$

> **Don't-care conditions ($\times$):** If certain input combinations can never occur (e.g., BCD has inputs 10–15 unused), mark them as don't-cares. They can be treated as 0 or 1, whichever gives a larger group.

Sequential Circuits

Sequential circuits have outputs that depend on both current inputs and stored state (past history). They require memory elements.

Table 8: Key differences between circuit types

Property	Combinational	Sequential
Memory	No	Yes
Output depends on	Current inputs	Inputs + stored state
Clock required	No	Usually yes
Examples	Adder, MUX	Counter, register, RAM
Described by	Truth table	State diagram / table

Combinational vs Sequential Comparison

Synchronous vs Asynchronous

Synchronous State changes only on the active clock edge. Easier to analyze and design. Dominant paradigm in modern digital design.

Asynchronous State changes immediately when inputs change. Faster but prone to glitches and hazards. Used in latches, FIFOs, and handshake logic.

Registers and Counters

A **register** is a group of n flip-flops storing an n-bit value. An n**-bit binary counter** cycles through $0 \to 1 \to 2 \to \cdots \to 2^n - 1 \to 0$, incrementing on each clock edge.

Flip-Flops

A flip-flop is a bistable memory element storing 1 bit. It is the fundamental building block of all sequential logic.

SR Latch (Level-Sensitive)

Table 9: SR latch characteristic table

S	R	Q^+	State
0	0	Q	Hold (no change)
1	0	1	Set
0	1	0	Reset
1	1	?	Forbidden

The SR latch can be built from two cross-coupled NAND or NOR gates. The forbidden state arises because $S = R = 1$ forces both outputs to the same value, and the final state becomes unpredictable when both return to 0 simultaneously.

D Flip-Flop

The most widely used flip-flop. Eliminates the forbidden state by connecting $\bar{D}$ to R.

$$Q^+ = D \quad \text{(on rising clock edge)}$$

Table 10: D flip-flop characteristic table

D	Q^+
0	0
1	1

JK Flip-Flop

Resolves the SR forbidden state by defining $J = K = 1$ as *toggle*:

$$Q^+ = J\bar{Q} + \bar{K}Q$$

Table 11: JK flip-flop characteristic table

J	K	Q^+	Operation
0	0	Q	Hold
1	0	1	Set
0	1	0	Reset
1	1	$\bar{Q}$	Toggle

T Flip-Flop

A JK flip-flop with $J = K = T$. Toggles on every clock edge when $T = 1$.

$$Q^+ = T \oplus Q$$

Used extensively in binary counter design.

Flip-Flop Excitation Tables

Excitation tables show what inputs are needed to achieve a desired state transition — essential for sequential circuit synthesis.

Table 12: Excitation tables for SR, D, JK, and T flip-flops

Transition		SR		D	JK		T
Q	Q^+	S	R		J	K	
0	0	0	×	0	0	×	0
0	1	1	0	1	1	×	1
1	0	0	1	0	×	1	1
1	1	×	0	1	×	0	0

Finite State Machines

A finite state machine (FSM) is a mathematical model of computation with a finite set of states Q, an input alphabet Σ, a transition function δ, an initial state q_0, and an output function.

$$\text{FSM} = (Q,\ \Sigma,\ \delta,\ q_0,\ \lambda)$$

Moore vs Mealy Machines

Table 13: Moore vs Mealy comparison

Property	Moore	Mealy
Output depends on	State only	State *and* input
Output changes	Only at clock edge	As soon as input changes
States typically needed	More	Fewer
Output stability	Glitch-free	May glitch with input
Notation on diagram	Output inside state circle	Output on transition arrow

Example: Traffic Light Controller (Moore)

States: {RED, GREEN, AMBER}

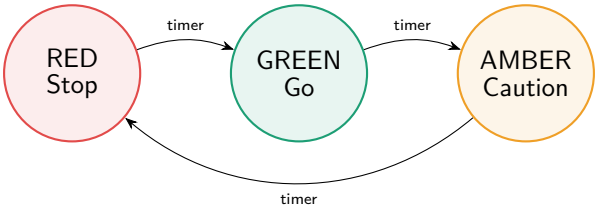

Figure 3: Traffic light Moore FSM state diagram

Table 14: Traffic light state transition table

Current State	Output (light)	Next State
RED	Stop	GREEN
GREEN	Go	AMBER
AMBER	Caution	RED

FSM Design Procedure

1. **Specify** the problem in plain language; identify inputs, outputs, and states.
2. **Draw** the state diagram with all transitions and outputs labelled.
3. **Encode** states in binary ($\lceil \log_2 |Q| \rceil$ flip-flops needed).
4. **Derive** the next-state and output equations from the state transition table.
5. **Simplify** using Boolean algebra or K-maps.
6. **Implement** using flip-flops and combinational logic.

Timing and Hazards

Propagation Delay

Every gate has a propagation delay t_p — the time from input change to output settling. In a combinational circuit, total delay = sum of delays along the longest path (critical path).

Setup and Hold Times

For a flip-flop to reliably capture data:

t_{setup} Data must be stable *before* the clock edge by at least t_{setup}.

t_{hold} Data must remain stable *after* the clock edge by at least t_{hold}.

The maximum clock frequency is:

$$f_{max} = \frac{1}{t_{cq} + t_{comb} + t_{setup}}$$

where t_{cq} is the clock-to-output delay of the flip-flop.

Hazards

A **hazard** is a spurious glitch in a combinational circuit output during input transitions.

Static-1 hazard Output should stay 1 but momentarily dips to 0.

Static-0 hazard Output should stay 0 but momentarily spikes to 1.

Dynamic hazard Output changes more than once when it should change exactly once.

> **Eliminating static hazards:** Add a *consensus term* that bridges the gap between adjacent K-map groups. The extra gate prevents the momentary glitch during switching.

Quick Reference

Powers of 2

n	2^n	n	2^n
0	1	8	256
1	2	10	1,024
2	4	16	65,536
3	8	20	1,048,576
4	16	24	16,777,216
8	256	32	4,294,967,296

Key Formulas

$$\text{2's complement :} \quad -N = \bar{N} + 1 \tag{1}$$

$$\text{MUX output :} \quad Y = \sum_i S_i \cdot I_i \tag{2}$$

$$\text{Decoder :} \quad Y_k = 1 \iff \text{input} = k \tag{3}$$

$$\text{D FF :} \quad Q^+ = D \tag{4}$$

$$\text{JK FF :} \quad Q^+ = J\bar{Q} + \bar{K}Q \tag{5}$$

$$\text{T FF :} \quad Q^+ = T \oplus Q \tag{6}$$

$$\text{Clock period :} \quad T \geq t_{cq} + t_{comb} + t_{setup} \tag{7}$$